Parte III · Escalabilidad (capítulos 14 a 23)

Notas de lectura — Vitillo, R. *Understanding Distributed Systems* (2.ª ed.) Sistemas Distribuidos · Escuela de Ingeniería de Sistemas e Informática · UIS

Propósito de la parte

Un sistema es **escalable** si puede atender más carga (usuarios, datos, peticiones) **agregando recursos** sin rediseñarse. Escalar verticalmente (una máquina más grande) tiene un techo; la Parte III trata del **escalado horizontal**: repartir el trabajo entre muchas máquinas.

Vitillo organiza las técnicas en **tres patrones**, que va aplicando a una aplicación web de ejemplo (*Cruder*, un CRUD sencillo que crece):

Patrón	Idea	Capítulos
Descomposición funcional	Separar la aplicación en componentes o servicios independientes	21 (microservicios), 22 (planos), 23 (mensajería)
Particionamiento	Dividir los datos o la carga en partes atendidas por nodos distintos	16, 17, 19
Replicación (duplicación)	Clonar datos o funcionalidad en varios nodos, y acercarlos al cliente	14, 15, 18, 19, 20

Cap.	Tema
14	Caché HTTP
15	Redes de distribución de contenido (CDN)
16	Particionamiento
17	Almacenamiento de archivos
18	Balanceo de carga en red
19	Almacenamiento de datos
20	Caché
21	Microservicios
22	Planos de control y planos de datos
23	Mensajería

Capítulo 14 · Caché HTTP

- Muchos recursos (imágenes, hojas de estilo, scripts) son **estáticos**: cambian poco y se piden mucho. HTTP permite que los clientes los **guarden en caché**.

- El servidor indica con el encabezado `Cache-Control: max-age=N` cuánto tiempo puede reutilizarse la respuesta sin preguntar.
- Al vencer, el recurso queda **obsoleto** (*stale*) y el cliente lo **revalida**: envía una petición condicional con `If-None-Match` y la `ETag` (identificador de versión) que recibió. Si no cambió, el servidor responde `304 Not Modified` sin cuerpo; si cambió, `200` con el nuevo contenido.
- Problema: un `max-age` largo impide ver cambios. Solución habitual: **URL versionadas** (p. ej., `app.3f9a2c.js`) con caché muy larga (o `immutable`): cada versión nueva es otro recurso.
- La caché HTTP es **eventualmente consistente** con el servidor.
- **Proxy inverso**: servidor intermedio frente a la aplicación que puede guardar en caché respuestas para **todos** los clientes, y además terminar TLS, comprimir, limitar tasa o balancear carga (p. ej., NGINX).

Capítulo 15 · Redes de distribución de contenido (CDN)

- Una CDN es una **red superpuesta** (*overlay*) de servidores distribuidos geográficamente que guarda contenido cerca de los usuarios. Es, en esencia, un **proxy inverso global** con caché.
- **Enrutamiento global:** un balanceo de carga por **DNS** (y/o **anycast**) dirige al cliente hacia el **punto de presencia (PoP)** más adecuado, considerando ubicación, salud y carga.
- Entre los PoP y el **origen**, la CDN usa rutas optimizadas y **conexiones persistentes** ya establecidas (con ventanas de congestión "calientes"), lo que reduce la latencia incluso para contenido no cacheable. Además, los *handshakes* TCP/TLS del cliente se hacen contra un servidor cercano (bajo RTT).
- **Caché:** varias capas: los servidores de borde (*edge*) y **cachés intermedias** que protegen al origen (*origin shield*), de modo que un fallo en el borde no siempre llega al origen.
- Dentro de un PoP, el contenido se **particiona** entre servidores (p. ej., con *hashing* consistente) para aprovechar la capacidad total de caché.
- Beneficios: menor latencia, menor carga en el origen y absorción de picos y ataques.

Capítulo 16 · Particionamiento

- Cuando los datos o la carga no caben en un nodo, se **dividen en particiones** (*shards*) repartidas entre nodos.
- Se necesita un **mapa partición → nodo**, que suele mantenerse en un servicio de coordinación (p. ej., ZooKeeper, etcd) y que consulta un **gateway** o el propio cliente para enrutar.
- Complicaciones: consultas y **transacciones que abarcan varias particiones** se vuelven costosas; hay que **rebalancear** cuando cambian los nodos o la carga.

Particionamiento por rango:

- Las claves se ordenan y se reparten en **rangos contiguos** (A–F, G–M…).
- Ventaja: las **consultas por rango** son eficientes.
- Riesgo: **puntos calientes** (*hot spots*), p. ej., claves basadas en fecha concentran todas las escrituras recientes en una partición. Se mitiga con prefijos o *splits*.

Particionamiento por *hash*:

- Se aplica una función *hash* a la clave y se reparte por valor de *hash*: distribución **uniforme**.
- Desventaja: se pierden las consultas por rango eficientes.

- Asignar con `hash(k) mod N` es problemático: al cambiar N, **casi todas** las claves cambian de nodo. Solución: ***hashing* consistente**: nodos y claves en un anillo; cada clave va al siguiente nodo en el sentido del anillo; al agregar o quitar un nodo solo se mueve una fracción ≈ 1/N de las claves.

Rebalanceo:

- **Particiones estáticas**: crear desde el inicio **muchas más particiones que nodos** y mover particiones completas entre nodos.
- **Particiones dinámicas**: **dividir** una partición cuando crece demasiado (o fusionar las pequeñas); se adapta a la carga real.

Capítulo 17 · Almacenamiento de archivos

- Para archivos grandes (imágenes, videos, respaldos) se usan **almacenes de objetos o *blobs*** gestionados (Amazon S3, Azure Blob Storage), en lugar de la base de datos.
- Vitillo describe la arquitectura de **Azure Storage** como caso de estudio de cómo se combinan los conceptos de las Partes II y III:
  - **Sellos** (*storage stamps*): clústeres de racks; la capacidad crece agregando sellos.
  - **Servicio de ubicación**: plano de control global que asigna cuentas a sellos y actualiza el **DNS** para enrutar las peticiones al sello correcto.
  - Dentro de un sello, tres capas:
    - **Capa de *front-end***: sin estado; autentica y enruta la petición a la partición correcta.
    - **Capa de particiones**: ofrece las abstracciones (blobs, tablas, colas); particiona por **rango** el espacio de nombres; un **administrador de particiones** asigna y rebalancea particiones entre servidores.
    - **Capa de *streams***: sistema de archivos distribuido de solo agregar; los datos se guardan en **extensiones** (*extents*) replicadas con **replicación en cadena**; un **administrador de *streams*** (plano de control con consenso) gestiona la configuración.
- Resultado: almacenamiento **fuertemente consistente**, altamente disponible y escalable.

Capítulo 18 · Balanceo de carga en red

- Para escalar un servicio **sin estado** se ejecutan varias réplicas detrás de un **balanceador de carga**, que reparte peticiones y oculta las réplicas tras una sola dirección.

Funciones de un balanceador:

- **Algoritmo de reparto:** *round-robin*, *hashing* consistente, menos conexiones o menor carga. La información de carga es imperfecta y puede causar efecto rebaño (todos eligen el servidor que parecía libre); una alternativa robusta es la **potencia de dos opciones aleatorias**: elegir dos servidores al azar y enviar al menos cargado.
- **Descubrimiento de servicios:** saber qué réplicas existen, mediante un **registro** dinámico (p. ej., etcd, Consul) en el que las instancias se registran.
- **Verificación de salud** (*health checks*): **pasiva** (detectar errores y *timeouts* en el tráfico real) o **activa** (consultar periódicamente un endpoint de salud); sacar de rotación las réplicas enfermas.

Tipos de balanceo:

- **Por DNS:** varias direcciones para un nombre. Simple, pero no conoce la salud de los servidores y las cachés retrasan los cambios.
- **Capa de transporte (L4):** opera sobre conexiones TCP/UDP con una **IP virtual (VIP)**; reparte **flujos** (p. ej., *hashing* consistente de la tupla de conexión) y mantiene el seguimiento de conexiones; puede usar **retorno directo del servidor** (*DSR*), en el que las respuestas no pasan por el balanceador. Muy eficiente, pero no ve el contenido HTTP.
- **Capa de aplicación (L7):** proxy HTTP que **termina** la conexión; enruta por ruta, encabezados o *cookies*; puede terminar TLS, reintentar, limitar tasa. Más funcionalidad, más costo. Puede desplegarse como ***sidecar*** junto a cada servicio (**malla de servicios**).
- El propio balanceador debe ser **escalable y tolerante a fallas** (varias instancias, p. ej., anunciadas con *anycast* o en par activo-pasivo).

Capítulo 19 · Almacenamiento de datos

- **Replicación:** un **líder** recibe las escrituras y **réplicas de lectura** (normalmente con replicación **asíncrona**) atienden lecturas; mejora disponibilidad y escalabilidad de lecturas, a costa de leer datos levemente atrasados. La replicación síncrona da durabilidad pero aumenta la latencia.
- **Particionamiento:** cuando las escrituras o el volumen superan un nodo, se particiona (manualmente o con bases de datos que lo gestionan).
- **NoSQL:** familias de almacenes (clave-valor, documentos, columnas anchas) que sacrifican flexibilidad de consulta (uniones, transacciones generales) a cambio de **escalabilidad horizontal** nativa.
- Ejemplo: **DynamoDB**: tablas de ítems con **clave de partición** (determina la partición por *hash*) y opcional **clave de ordenamiento** (orden dentro de la partición, consultas por rango). Índices secundarios **locales** (misma clave de partición) y **globales** (otra clave de partición).
- En NoSQL se **diseña a partir de los patrones de acceso**: primero se definen las consultas y luego el modelo, a menudo **desnormalizado** (incluso una sola tabla con varias entidades), al revés que en el modelo relacional.

Capítulo 20 · Caché

- Una caché guarda resultados costosos de obtener para servirlos más rápido. Su eficacia se mide con la **tasa de aciertos** (*hit ratio*), que depende de cuántas veces se repite el acceso a los mismos datos.
- **Políticas: expiración** (TTL) para no servir datos demasiado viejos y **desalojo** cuando se llena (p. ej., **LRU**, el menos usado recientemente).
- **Caché local (en proceso):** rápida y simple, pero cada instancia tiene la suya → duplicación, inconsistencia entre instancias, y caché **fría** tras cada despliegue o reinicio; si muchas instancias pierden la caché a la vez, la base de datos recibe una avalancha (*thundering herd*).
- **Caché externa (compartida):** servicio dedicado (Redis, Memcached), particionado (p. ej., *hashing* consistente) y replicado; todas las instancias comparten los datos. Pero es **otra dependencia** que puede fallar: si cae, toda la carga va a la base de datos, que quizá no la soporte. El sistema debe estar preparado (p. ej., descartar carga).
- **Patrones de acceso:**
  - ***Cache-aside*** (caché lateral): la aplicación consulta la caché; si falla, va a la base de datos y luego llena la caché.
  - ***Inline*** (*read-through* / *write-through*): la aplicación habla solo con la caché, que se encarga de la base de datos.

- Vitillo advierte: una caché no debe ser la **muleta** que oculta que la base de datos no aguanta la carga; si la caché se vacía, el sistema debe seguir en pie.

Capítulo 21 · Microservicios

- **Descomposición funcional**: dividir una aplicación en servicios **independientes**, cada uno con su base de datos, que se comunican por API. Permite que equipos distintos **desarrollen, desplieguen y escalen** sus servicios de forma independiente.
- **Inconvenientes** que señala el libro:
  - proliferación de tecnologías;
  - **comunicación** por red (latencia, fallas parciales, versionado de API);
  - **acoplamiento** oculto si los límites están mal trazados (cambios que obligan a desplegar varios servicios a la vez: un "monolito distribuido");
  - aprovisionamiento de recursos, **pruebas** de extremo a extremo y **operación** más difíciles;
  - **consistencia eventual** entre servicios (sin transacciones locales compartidas → sagas).
- Recomendación: **empezar con un monolito** bien modularizado y separar servicios cuando los límites estén claros y el beneficio supere el costo.
- **API *gateway***: punto de entrada único que se ubica frente a los servicios internos:
  - **enrutamiento** de peticiones al servicio adecuado;
  - **composición**: combinar respuestas de varios servicios en una;
  - **traducción** entre protocolos o formatos (p. ej., GraphQL ↔ REST, HTTP ↔ gRPC);
  - **funciones transversales**: autenticación y autorización (p. ej., *tokens* JWT), limitación de tasa, caché, registro.
  - Riesgo: convertirse en cuello de botella o concentrar lógica que acopla a todos los equipos.

Capítulo 22 · Planos de control y planos de datos

- Un servicio de gran escala suele separarse en:
  - **plano de datos**: procesa cada petición individual; está en el **camino crítico**; debe ser rápido y altamente disponible;
  - **plano de control**: gestiona configuración, metadatos y ciclo de vida (crear recursos, asignar particiones, desplegar); no está en el camino crítico de cada petición.
- Ejemplos: el balanceador reparte tráfico (datos) mientras otro componente actualiza la lista de réplicas (control); en Azure Storage, el administrador de *streams* (control) frente a los servidores de extensiones (datos).
- **Desequilibrio de escala:** el plano de datos suele ser mucho más grande; si muchos nodos de datos consultan al plano de control a la vez (p. ej., al reiniciar todos), pueden **saturarlo**. Mitigaciones: que el plano de control **empuje** cambios, cachés de configuración y limitación.
- **Estabilidad estática** (*static stability*): si el plano de control falla, el plano de datos debe **seguir funcionando con la última configuración conocida**, aunque no pueda aplicar cambios.
- **Teoría de control:** el plano de control funciona como un **lazo cerrado**: **monitorea** el estado del sistema, lo **compara** con el estado deseado y **actúa** para corregir la diferencia (reconciliación; p. ej., los controladores de Kubernetes, el autoescalado).

Capítulo 23 · Mensajería

- En lugar de peticiones síncronas, los servicios pueden comunicarse **asíncronamente** a través de un **canal de mensajes** (cola o *broker*: SQS, RabbitMQ, Kafka).
- **Beneficios:** desacopla emisor y receptor (no necesitan estar disponibles a la vez), **nivela la carga** (la cola absorbe picos), permite procesar en lote y escalar consumidores.
- **Tipos de canal: punto a punto** (cada mensaje lo procesa **un** consumidor) y **publicación-suscripción** (cada mensaje llega a **todos** los suscriptores).
- **Patrones:** mensajería **unidireccional**; **petición-respuesta** (dos canales e identificadores de correlación); **difusión**.
- **Garantías** que varían entre implementaciones: orden (p. ej., Kafka ordena por partición), entrega, durabilidad.
- **Exactamente una vez** no es posible en general: los *brokers* ofrecen **al menos una vez** (tras un fallo, el mensaje se reentrega cuando vence su **tiempo de visibilidad** si no se confirmó). Se logra **procesamiento efectivamente una vez** haciendo a los consumidores **idempotentes** o deduplicando.
- **Fallas:** un mensaje que siempre falla (**mensaje envenenado**) bloquearía la cola o se reintentaría eternamente; tras N intentos se mueve a una **cola de mensajes muertos** (*dead letter queue*) para análisis.
- **Acumulación (*backlog*):** si los productores superan a los consumidores, la cola crece; hay que **monitorear** la antigüedad y el tamaño de la cola, y escalar consumidores.

Conceptos clave (glosario)

Término	Definición breve
Escalado horizontal	Agregar máquinas en lugar de agrandar una
`Cache-Control` / `ETag`	Vigencia de una respuesta / identificador de versión para revalidar
CDN	Red superpuesta de cachés y proxies cercanos al usuario
Hashing consistente	Asignación en anillo que mueve ≈ 1/N claves al cambiar nodos
Punto caliente	Partición que recibe una fracción desproporcionada de la carga
Balanceador L4 / L7	Reparte conexiones / peticiones de aplicación
Potencia de dos opciones	Elegir el menos cargado entre dos servidores aleatorios
Réplica de lectura	Copia (asíncrona) que atiende lecturas
Clave de partición / ordenamiento	Determinan partición y orden en DynamoDB
Cache-aside	La aplicación gestiona caché y base de datos
API *gateway*	Punto de entrada que enruta, compone y aplica funciones transversales
Estabilidad estática	El plano de datos sigue funcionando sin el plano de control
Cola de mensajes muertos	Destino de mensajes que fallan repetidamente

---

Preguntas guía para la lectura

1. ¿Cuáles son los tres patrones de escalabilidad y qué capítulos aplican cada uno?
2. ¿Cómo combina HTTP la expiración y la revalidación para servir recursos desde caché?
3. ¿Por qué una CDN reduce la latencia incluso para contenido que no se puede cachear?
4. ¿Qué se gana y qué se pierde al particionar por rango o por *hash*?
5. ¿Qué diferencia hay entre un balanceador de capa 4 y uno de capa 7?
6. ¿Por qué una caché externa puede convertirse en un riesgo para la disponibilidad?
7. ¿Cuándo no conviene dividir una aplicación en microservicios?
8. ¿Qué significa estabilidad estática?
9. ¿Por qué la mensajería ofrece "al menos una vez" y cómo se logra el efecto "exactamente una vez"?

Conexión con otras partes

- **Parte I:** DNS (balanceo global, servicio de ubicación de Azure), TLS (terminación en CDN y balanceadores), APIs (gateway).
- **Parte II:** replicación en cadena (Azure Storage), consistencia eventual (cachés, réplicas de lectura), sagas y *outbox* (microservicios y mensajería), idempotencia.
- **Parte IV (resiliencia):** *timeouts*, reintentos, limitación de carga y degradación ante fallas de cachés, planos de control y dependencias.